EXPLORATORY DATA ANALYSIS

Chapter 21

Automated EDA and Data Profiling

Using automation to accelerate investigation without surrendering analytical judgment

Course tools: Python, NumPy, Pandas, Matplotlib's object-oriented API, Seaborn, and selected profiling tools.

This chapter treats automated EDA as an assistant to a question-driven workflow. We will inspect reports, understand their statistics, validate their warnings, and produce reproducible outputs.

Learning objectives

• Explain what automated EDA and data profiling can summarize and what they cannot establish.

• Distinguish schema, quality, univariate, bivariate, multivariate, and temporal profiling.

• Build a manual profiling baseline with Pandas before using an automated tool.

• Use profiling tools such as ydata-profiling and Sweetviz appropriately.

• Configure reports for larger datasets and understand sampling and performance trade-offs.

• Interpret warnings about missingness, cardinality, duplicates, skew, correlations, and constant columns.

• Validate automated findings against domain knowledge and source records.

• Create a reproducible profiling workflow and communicate limitations.

Central principle: an automated report is a map of possible questions, not a substitute for answering them.

1. What is automated EDA?

Automated EDA uses software to generate descriptive summaries, visualizations, relationship scans, and data-quality warnings from a dataset. A profiler can rapidly inspect hundreds of columns and reveal patterns that would be tedious to calculate manually.

Typical outputs include column types, missingness, distinct-value counts, descriptive statistics, frequency tables, histograms, correlations, duplicate indicators, and warnings. Some tools also provide interactions, HTML reports, or comparisons between datasets.

Automation is especially useful for an initial scan, repeated client workflows, data intake checks, and documenting a consistent baseline. It is less reliable when column meaning, measurement rules, sampling, or domain constraints are unknown.

2. Review of earlier chapters (Chapters 1–19)

This review excludes Chapter 20, which is reviewed separately below.

• **Ch. 1 — What is EDA?** EDA is iterative investigation guided by context and questions.

• **Ch. 2 — Understanding the dataset:** identify observational units, grain, metadata, and variable types.

• **Ch. 3 — Data quality:** audit completeness, validity, consistency, uniqueness, accuracy, and timeliness.

• **Ch. 4 — Numerical variables:** inspect location, spread, distribution, and unusual values.

• **Ch. 5 — Categorical variables:** inspect frequencies, rare levels, cardinality, and denominators.

• **Ch. 6 — Date and time:** validate parsing, time zones, temporal coverage, and intervals.

• **Ch. 7 — Descriptive statistics:** choose robust and conventional summaries appropriately.

• **Ch. 8 — Distributions:** understand histograms, ECDFs, KDE, density, and distribution form.

• **Ch. 9 — Shape:** interpret skewness, kurtosis, and quantiles with care.

• **Ch. 10 — Visualizing distributions:** choose plots that reveal rather than conceal structure.

• **Ch. 11 — Visualizing relationships:** inspect form, strength, nonlinearity, and confounding.

• **Ch. 12 — Multivariate visualization:** use facets, heatmaps, and encodings responsibly.

• **Ch. 13 — Association:** match measures to variable types; association is not causation.

• **Ch. 14 — Covariance and dependence:** distinguish correlation from independence.

• **Ch. 15 — Grouped analysis:** make denominators and aggregation rules explicit.

• **Ch. 16 — Outlier detection:** flags identify candidates for investigation, not automatic deletions.

• **Ch. 17 — Investigating unusual observations:** validate source records and compare relevant peers.

• **Ch. 18 — High-dimensional EDA:** organize variables and avoid overinterpreting large scans.

• **Ch. 19 — Time series EDA:** respect cadence, temporal gaps, seasonality, and collection changes.

3. Review of Chapter 20 — Sampling, Missingness Patterns, and Data Collection Bias

Chapter 20 examined target populations, sampling frames, probability and non-probability selection, coverage, nonresponse, survivorship, measurement bias, missingness mechanisms, selection effects, join-induced loss, and sensitivity analysis.

> **Expected reasoning for the Chapter 20 assignment/project:** a sound collection audit defines the target population and analytical sample, describes the capture mechanism, quantifies missingness with explicit denominators, inspects subgroup/time differences, audits joins and filters, and separates observed evidence from hypotheses about bias. If no assignment was submitted, this is a reasoning guide rather than a grading result.

A profiler can show that 18% of a column is missing. It cannot, by itself, tell you whether those values are missing because of skip logic, nonresponse, a system outage, or a selection process.

4. The layers of a data profile

Layer	Typical questions	Example outputs
Schema	What columns and types exist?	Names, inferred types, dimensions
Univariate	What does each variable look like?	Mean, quantiles, frequencies, histograms
Quality	What may be malformed or incomplete?	Missingness, duplicates, constants
Relationships	Which variables move together?	Correlation and contingency summaries
Multivariate	Are there broader patterns?	Pair plots, grouped patterns, redundancy
Temporal	How does data behave over time?	Coverage, gaps, changing distributions
Provenance	How did these records enter the dataset?	Source, extraction date, filters, frame

Most general-purpose profilers focus on schema, univariate summaries, basic quality, and selected relationships. Temporal and provenance analysis often require custom work.

5. Start with a manual baseline

Before opening a report, create a small reproducible baseline. This helps you check whether a profiler's inferred types and statistics make sense.

```
import numpy as np
import pandas as pd

def basic_profile(df):
    profile = pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "missing_n": df.isna().sum(),
        "missing_rate": df.isna().mean(),
        "unique_n": df.nunique(dropna=True),
        "memory_bytes": df.memory_usage(
            index=False, deep=True
        )
    })
    profile["observed_n"] = len(df) - profile["missing_n"]
    profile["unique_rate"] = (
        profile["unique_n"] / profile["observed_n"].replace(0, np.nan)
    )
    return profile.sort_values(
        ["missing_rate", "unique_n"],
        ascending=[False, False]
    )

profile = basic_profile(df)
profile
```

This baseline is intentionally simple. It does not decide whether a column is valid, whether a high-cardinality field is an identifier, or whether missingness is acceptable. Those decisions require context.

6. Schema inference and semantic types

A profiler may infer integer, float, boolean, datetime, text, or categorical types from observed values. Inference is a useful guess, not a data dictionary.

• A numeric-looking customer ID is usually an identifier, not a quantity.

• A date stored as text may be parsed incorrectly if formats are mixed.

• An integer-coded category (for example, 1, 2, 3) may not have arithmetic meaning.

• A boolean-like column with values 0/1 may encode a status, a count, or a measurement.

• A free-text column may contain structured codes or mixed-language categories.

Always compare inferred types with metadata and the meaning of the variable. Explicitly convert types only after checking parsing failures and preserving original values where needed.

7. Univariate profiling

7.1 Numerical columns

A profile may report count, mean, standard deviation, minimum, quantiles, maximum, skewness, and a histogram. Inspect whether the statistics are meaningful for the measurement scale and whether a small number of extreme values distort the display.

7.2 Categorical and text columns

Frequency tables and distinct counts can reveal rare levels, inconsistent spelling, unexpected blanks, and identifier-like fields. A profiler may truncate long frequency tables; inspect the full value distribution when necessary.

7.3 Dates and times

Date inference does not establish the correct timezone, event-time meaning, or expected cadence. Validate ranges, parsing failures, duplicate timestamps, and gaps separately.

7.4 Quantile and histogram settings

Different tools may use different quantile interpolation rules, histogram binning rules, or missing-value conventions. Small numeric discrepancies do not automatically indicate an error. Record settings when exact reproducibility matters.

8. Quality warnings: useful prompts, not verdicts

Warning	Why it may matter	Why it may be harmless
High missingness	Could reflect nonresponse or capture failure	Field is optional or structurally not applicable
High cardinality	May be an ID or uncontrolled category	Expected for names, URLs, or transaction keys
Constant column	May add no analytical information	Could be a required status in this sample
Near-constant column	May hide rare but important events	Rare event may be central to the question
Duplicate rows	Could indicate repeated ingestion	Rows may be legitimate repeated events
High correlation	Possible redundancy or data leakage	Variables may be mathematically related by design
Skew / outliers	May affect summaries and plots	Valid values may naturally be highly skewed

A warning should lead to a question: what is the column's role, how was it collected, and what does the domain say? Do not automatically drop columns or records because a report labels them suspicious.

9. Relationship scans and their limitations

Many profilers calculate pairwise correlations for numeric variables and selected associations for categorical variables. Such scans can reveal obvious redundancy, but they also create a multiple-comparisons problem: with p variables there are $p(p–1)/2$ numeric pairs.

A large scan will produce striking values by chance or through indirect structure. Correlation can be driven by shared time trends, common denominators, subgroup composition, outliers, or mathematically coupled variables. The report usually does not know which explanation is appropriate.

• Check sample size for each pair; pairwise missingness can vary.

• Inspect scatterplots for important or surprising associations.

• Use measures appropriate to the variable types and relationship shape.

• Consider confounding and subgroup structure.

• Do not interpret a correlation heatmap as a causal map.

10. Automated EDA tools in the Python ecosystem

10.1 ydata-profiling

The ydata-profiling package generates a detailed profile report with variable summaries, interactions, correlations, missingness information, and alerts. It is useful for broad first-pass inspection and HTML report generation. The exact API and options can change across releases, so verify the installed version's documentation.

```
# Install in your environment:
# %pip install -U ydata-profiling

from ydata_profiling import ProfileReport

profile_report = ProfileReport(
    df,
    title="Dataset EDA Profile",
    explorative=True,
    minimal=False
)
profile_report.to_file("eda_profile.html")
```

For very wide or large datasets, a full explorative report may be slow or overwhelming. Start with a representative subset or a minimal report, then profile selected columns in detail.

10.2 Sweetviz

Sweetviz creates visual HTML reports and can compare datasets, such as train versus test or one group versus another. It is particularly useful for quick distribution comparisons. As with every tool, verify supported data types and current package compatibility.

```
# Install in your environment:
# %pip install -U sweetviz

import sweetviz as sv

report = sv.analyze(df)
report.show_html("sweetviz_report.html", open_browser=False)

# Compare two subsets when the split is meaningful.
comparison = sv.compare(
    [train_df, "Train"],
    [test_df, "Test"]
)
comparison.show_html(
    "train_test_comparison.html",
    open_browser=False
)
```

A train/test comparison can reveal distribution differences; it does not prove that a split is valid or that there is no leakage. For time-dependent or grouped data, the split design must respect the data structure.

10.3 Other tools and built-in summaries

Pandas describe(), missingness libraries, notebook visualizations, and database-side profiling can all support profiling. Select tools based on the workflow, output format, dataset size, and maintenance needs—not the number of charts they generate.

11. Large datasets: sampling, performance, and reproducibility

Profiling can consume substantial memory and time, especially when text columns have many unique values or pairwise relationships are computed across many variables.

• Inspect dimensions, dtypes, and memory before running an expensive report.

• Use a seeded sample for exploratory visualization when appropriate.

• Preserve rare categories and important subgroups when sampling.

• Do not use a convenience sample to claim population representativeness.

• Profile high-value column groups separately when the dataset is very wide.

• Record row counts and sampling rules so the report's scope is clear.

```
# Reproducible simple random sample for exploratory profiling.
sample_n = min(20_000, len(df))
profile_sample = df.sample(
    n=sample_n,
    random_state=42
)

# Record what was profiled.
profile_metadata = {
    "source_rows": len(df),
    "profiled_rows": len(profile_sample),
    "sample_seed": 42,
    "sampling_method": "simple random sample without replacement"
}
```

A random sample can omit rare but important events. For rare categories or anomalies, use a targeted supplementary inspection and label it clearly as such.

12. Missingness profiles are not missingness explanations

Automated reports can show missing counts, rates, and co-occurrence. They generally cannot determine MCAR, MAR, or MNAR from a table alone. Combine the report with questionnaire logic, source-system logs, collection documentation, and subgroup/time analysis.

```
missing_summary = pd.DataFrame({
    "missing_n": df.isna().sum(),
    "missing_rate": df.isna().mean(),
    "unique_observed": df.nunique(dropna=True)
}).sort_values("missing_rate", ascending=False)

# Missingness rates by a collection-related subgroup.
by_source = (
    df.assign(value_missing=df["value"].isna())
      .groupby("source_system", dropna=False)
      ["value_missing"]
      .agg(["size", "mean"])
      .rename(columns={"size": "n", "mean": "missing_rate"})
)
```

The subgroup pattern may suggest a collection mechanism, but it does not prove the mechanism. Verify with operational evidence.

13. Compare datasets and monitor drift

Profiling becomes especially useful when comparing two datasets: a new extract versus a previous extract, one source versus another, or a training split versus a test split. Compare schema, row counts, missingness, category frequencies, quantiles, and key distributions.

• Schema drift: columns appear, disappear, or change type.

• Distribution drift: values or category proportions change.

• Coverage drift: different entities or time periods are represented.

• Quality drift: missingness, invalid values, or duplicates increase.

• Process drift: extraction, instrumentation, or business rules change.

A difference is not automatically a defect. It may reflect a real population or process change. The purpose of comparison is to detect and investigate change.

14. A reproducible profiling workflow

• Define the analytical question, target population, unit of observation, and data scope.

• Record source, extraction timestamp, filters, row count, and version or checksum where feasible.

• Inspect schema and memory; validate types against metadata.

• Run a manual baseline profile.

• Generate an automated report with documented settings and sampling rules.

• Review alerts and prioritize those relevant to the analytical question.

• Validate important findings with targeted code, plots, and source records.

• Compare missingness and distributions across meaningful groups and time.

• Document unresolved questions, limitations, and decisions.

• Save the notebook, report, environment details, and a concise executive summary.

15. A reusable profile-and-alert baseline

```
def profile_baseline(df):
    numeric = df.select_dtypes(include="number")
    categorical = df.select_dtypes(
        include=["object", "category", "bool"]
    )

    profile = pd.DataFrame(index=df.columns)
    profile["dtype"] = df.dtypes.astype(str)
    profile["missing_n"] = df.isna().sum()
    profile["missing_rate"] = df.isna().mean()
    profile["unique_n"] = df.nunique(dropna=True)

    numeric_summary = numeric.describe(
        percentiles=[0.01, 0.25, 0.50, 0.75, 0.99]
    ).T

    categorical_summary = pd.DataFrame({
        "top_value": categorical.apply(
            lambda s: s.mode(dropna=True).iloc[0]
            if not s.mode(dropna=True).empty else np.nan
        ),
        "top_frequency": categorical.apply(
            lambda s: s.value_counts(dropna=True).iloc[0]
            if not s.value_counts(dropna=True).empty else 0
        )
    })

    alerts = {
        "all_missing": profile.index[
            profile["missing_rate"].eq(1)
        ].tolist(),
        "constant": profile.index[
            profile["unique_n"].le(1)
        ].tolist(),
        "high_missing_over_40pct": profile.index[
            profile["missing_rate"].gt(0.40)
        ].tolist()
    }

    return {
        "overview": profile,
        "numeric_summary": numeric_summary,
        "categorical_summary": categorical_summary,
        "alerts": alerts
    }

baseline = profile_baseline(df)
```

This function is a transparent starting point, not a replacement for a mature profiler. It deliberately produces alerts rather than automatically changing the dataset. In a production workflow, add explicit rules for identifiers, dates, domain constraints, and source-specific expectations.

16. Common mistakes

• Treating an automated report as a complete EDA deliverable.

• Assuming inferred types match semantic variable roles.

• Treating warnings as proof of data errors.

• Automatically deleting high-cardinality, constant, or highly correlated columns.

• Confusing a missingness pattern with proof of MCAR, MAR, or MNAR.

• Reading pairwise correlations without checking sample sizes and plots.

• Profiling a sample without documenting its selection and limitations.

• Comparing train and test data without considering the split design.

• Ignoring time zones, temporal coverage, and source-system changes.

• Failing to record tool version, configuration, and input data version.

• Producing a large report without a concise explanation of the findings that matter.

17. Summary

• Automated EDA accelerates descriptive scanning, quality checks, and report generation.

• A profile summarizes what is present; it cannot explain why the data looks that way.

• Manual baselines make automated outputs easier to validate.

• Schema inference, alerts, and relationship scans require semantic and domain review.

• Large-data profiling needs explicit sampling and reproducibility choices.

• Missingness reports do not identify missingness mechanisms by themselves.

• Dataset comparisons help detect schema, distribution, coverage, and process drift.

• Professional EDA combines automation with targeted investigation and clear communication.

18. Assignment

Part A — Concepts

• Explain the difference between a data profile and a complete EDA investigation.

• Give two examples where a profiler's warning could be harmless, and two where it could reveal a serious issue.

• Why can an automated correlation scan produce misleading conclusions?

• Why does a missingness report not establish MCAR, MAR, or MNAR?

• List the metadata you would save to make an automated report reproducible.

Part B — Manual profile

• Build a schema and memory overview.

• Create a column-level profile with dtype, missing count/rate, unique count, and observed count.

• Summarize numerical and categorical variables separately.

• Flag all-missing, constant, near-constant, and high-cardinality columns for review.

Part C — Automated report

• Generate a ydata-profiling or Sweetviz report.

• Record package version, configuration, input dimensions, and any sampling method.

• Choose at least five alerts or surprising outputs and validate them manually.

• Inspect at least two important relationships with appropriate plots.

Part D — Comparison

If you have two meaningful dataset versions or groups, compare schema, missingness, distributions, and category frequencies. Explain whether each difference appears to be a data issue, process change, or plausible real-world change.

Part E — Reflection

Which important questions about your dataset could the automated tool not answer without domain knowledge or collection-process information?

19. Coding project — Reproducible Automated EDA Pipeline

Extend your cumulative EDA master notebook. Build a small reusable pipeline that produces both machine-readable summaries and a human-readable report.

• Create a dataset intake cell that records dimensions, columns, dtypes, source, and extraction time.

• Generate the manual baseline profile and quality alerts.

• Create an automated HTML report using ydata-profiling or Sweetviz.

• Document the tool version, settings, and any sampling strategy.

• Validate at least five alerts with targeted Pandas calculations or visualizations.

• Include a missingness-by-subgroup or missingness-over-time investigation.

• Inspect selected relationships and explain why they matter.

• If possible, compare two data extracts and identify schema or distribution drift.

• Write a concise executive summary with findings, caveats, and recommended next investigations.

• Keep original data unchanged; do not let the profiler silently become a cleaning pipeline.

Acceptance criteria: the workflow can be rerun; its inputs and settings are documented; outputs are saved; warnings are investigated rather than blindly accepted; and the final summary distinguishes automated observations from analyst interpretation.

Suggested deliverables: cumulative notebook, HTML profiling report, machine-readable profile table, selected validation plots, environment/version note, and one-page findings summary.

20. Preview of Next Chapter

Chapter 22 — EDA for Different Data Domains

We will adapt the EDA workflow to different kinds of real-world datasets: business and customer data, transactions, education, healthcare-style records, web and product analytics, operational logs, and other domains. We will focus on how the observational unit, data-generating process, domain constraints, and useful questions change from one domain to another.

Boundary: the goal is to adapt investigative thinking and reporting—not to teach domain-specific professional practice or build predictive models.